

LAB ASSIGNMENT REPORT

MODULE 4 – ROBOT FRAMEWORK

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Exact Syllabus-wise Practical / Lab Report

Module 4 Lab Coverage

This report follows the Module 4 Robot Framework curriculum in the uploaded course sheet: Introduction, environment setup, RIDE, keyword/data-driven cases, command-line execution, Selenium Library, execution options, Sauce Labs, Jenkins scheduling, user-defined keywords, resource files, setup/teardown, Page Objects and Gherkin readability.

Recommended installation: `python -m pip install robotframework robotframework-seleniumlibrary selenium`

Note: Browser automation examples assume Chrome/ChromeDriver or Selenium Manager is available.

4.1 Robot Framework Introduction and Environment Setup

The first practical section introduces Robot Framework syntax, test cases, keywords, settings and execution. Robot Framework uses plain-text tabular syntax and is extensible through libraries such as SeleniumLibrary.

Experiment 1 – Basic Robot Framework Test Case

Aim: To create and execute a basic Robot Framework test case using built-in keywords.

Procedure

1. Create a file named basic.robot.
2. Define Settings and Test Cases sections.
3. Use built-in Log and Should Be Equal keywords.
4. Run the test with robot basic.robot.
5. Observe the generated log.html and report.html files.

Robot Framework Source

```
*** Settings ***
Documentation    Basic Robot Framework demonstration

*** Test Cases ***
Verify Basic Calculation
    ${result}=    Evaluate    10 + 20
    Should Be Equal As Integers    ${result}    30
    Log    Calculation verified successfully
```

Expected Result: The test passes and Robot Framework generates report.html, log.html and output.xml.

Conclusion: Robot Framework test cases are readable and can be composed from reusable keywords.

Experiment 2 – Environment Setup and Selenium Library

Aim: To configure SeleniumLibrary and open a browser from a Robot Framework test.

Procedure

1. Install Robot Framework and SeleniumLibrary.
2. Create browser.robot.
3. Import SeleniumLibrary.
4. Open a browser to a test URL.
5. Close the browser.
6. Run the test from the command line.

Robot Framework Source

```
*** Settings ***
Library    SeleniumLibrary

*** Test Cases ***
Open Test Website
    Open Browser    https://www.selenium.dev/    chrome
    Title Should Be    Selenium
    [Teardown]    Close All Browsers
```

Expected Result: The Chrome browser opens the Selenium website, the title is validated, and the browser closes during teardown.

Conclusion: SeleniumLibrary extends Robot Framework with browser automation keywords.

4.2 Keyword and Data-Driven Test Cases

Robot Framework supports user-defined keywords and data-driven testing. Test data can be represented in variables, templates or external resource files.

Experiment 3 – User-Defined Keywords

Aim: To create reusable keywords that hide low-level browser operations.

Procedure

1. Create a reusable Open Login Page keyword.
2. Create a reusable Verify Page Title keyword.
3. Call the keywords from a test case.
4. Keep the test case readable.

Robot Framework Source

```
*** Settings ***
Library    SeleniumLibrary

*** Keywords ***
Open Selenium Home Page
    Open Browser    https://www.selenium.dev/    chrome

Verify Selenium Title
    Title Should Be    Selenium

*** Test Cases ***
Reusable Keyword Demo
    Open Selenium Home Page
    Verify Selenium Title
    [Teardown]    Close All Browsers
```

Expected Result: The test passes while the browser operations are encapsulated in reusable keywords.

Conclusion: User-defined keywords improve readability and reduce duplication.

Experiment 4 – Data-Driven Test Case

Aim: To execute the same Robot Framework test with multiple data values.

Procedure

1. Create a test template keyword.
2. Define multiple rows of test data.
3. Use the [Template] setting.
4. Execute the same keyword for every data row.
5. Review the results.

Robot Framework Source

```
*** Settings ***
Library    String

*** Test Cases ***
Validate Multiple Values
    [Template]    Validate Text
    Robot Framework
    Selenium
    Python Automation

*** Keywords ***
Validate Text
    [Arguments]    ${text}
    Should Not Be Empty    ${text}
    Log    Validated value: ${text}
```

Expected Result: Robot Framework executes the template keyword for each data row.

Conclusion: Data-driven execution allows one test design to cover multiple input combinations.

4.3 Command Line, Browsers and Execution Options

Robot Framework can be run from PyCharm, a command window, batch files and scheduled tasks. SeleniumLibrary supports browser automation, while execution options control suites, tests, tags and output locations.

Experiment 5 – Command-Line Execution and Test Selection

Aim: To execute selected Robot Framework tests and organize generated output.

Procedure

1. Create a suite containing multiple tests.
2. Run the complete suite using robot.
3. Run one test using --test.
4. Set an output directory using -d.
5. Review report.html and log.html.

Robot Framework Source

```
*** Test Cases ***
Smoke Test
    Log    Smoke test executed

Regression Test
    Log    Regression test executed

# Examples:
# robot suite.robot
# robot --test "Smoke Test" suite.robot
# robot -d results suite.robot
```

Expected Result: The selected test executes and the output files are stored in the specified directory.

Conclusion: Command-line options make Robot Framework suitable for local and CI execution.

Experiment 6 – Browser Navigation and Form Interaction

Aim: To automate basic browser navigation and form interaction using SeleniumLibrary.

Procedure

1. Open a browser.

2. Navigate to a page.
3. Use Input Text and Click Element.
4. Validate visible text.
5. Close the browser.

Robot Framework Source

```
*** Settings ***
Library    SeleniumLibrary

*** Test Cases ***
Browser Interaction
    Open Browser    https://www.selenium.dev/    chrome
    Wait Until Page Contains    Selenium
    Click Link    Documentation
    Wait Until Page Contains    Selenium Documentation
    [Teardown]    Close All Browsers
```

Expected Result: The browser navigates to the Documentation page and the expected text is found.

Conclusion: SeleniumLibrary provides high-level keywords for common browser actions.

4.4 Sauce Labs, Jenkins and Scheduling

The syllabus includes running browser tests on Sauce Labs and scheduling execution through Jenkins. These are normally configured with remote WebDriver capabilities and CI commands.

Experiment 7 – Remote Browser Execution Pattern

Aim: To demonstrate the Robot Framework structure used for remote browser execution such as Sauce Labs.

Procedure

1. Create remote WebDriver capabilities according to the provider documentation.
2. Store credentials in environment variables rather than source code.
3. Use Open Browser with the remote URL and desired capabilities.
4. Execute the suite and close the session.

Robot Framework Source

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${SAUCE_URL}    ${SAUCE_URL}
${SAUCE_USER}    ${SAUCE_USERNAME}
${SAUCE_KEY}    ${SAUCE_ACCESS_KEY}

*** Test Cases ***
Remote Browser Test
    [Documentation]    Remote execution pattern
    # Provider-specific capabilities should be supplied
    # according to the current Sauce Labs configuration.
    Log    Remote browser configuration loaded
```

Expected Result: The test framework loads provider configuration without hard-coding credentials.

Conclusion: Cloud execution should use secure credentials and provider-specific WebDriver capabilities.

Experiment 8 – Jenkins Command and Scheduled Execution

Aim: To prepare Robot Framework execution for Jenkins or another CI scheduler.

Procedure

1. Install Robot Framework on the Jenkins agent.
2. Place the test suite in the workspace.
3. Execute robot from a Jenkins build step.
4. Archive Robot Framework reports.
5. Schedule the Jenkins job using its build triggers.

Robot Framework Source

```
# Jenkins Execute Shell / Windows batch example

robot -d results tests/

# The Jenkins job can then archive:
# results/output.xml
# results/log.html
# results/report.html

# Example command for a specific tag:
# robot --include smoke -d results tests/
```

Expected Result: Jenkins executes the suite and stores the Robot Framework result artifacts.

Conclusion: Robot Framework integrates naturally with CI pipelines and scheduled execution.

4.5 Readability, Resource Files, Setup/Teardown and Page Objects

The final syllabus section focuses on making test automation maintainable through user-defined keywords, resource files, setup/teardown, Page Objects and Gherkin-style readability.

Experiment 9 – Resource File and Setup/Teardown

Aim: To move reusable keywords into a resource file and apply suite/test setup and teardown.

Procedure

1. Create resources/common.resource.
2. Place reusable keywords in the resource file.
3. Import the resource.
4. Use Suite Setup and Suite Teardown.
5. Execute the suite.

Robot Framework Source

```
# resources/common.resource

*** Keywords ***
Log Test Start
    Log    Test started

Log Test End
    Log    Test finished

# suite.robot

*** Settings ***
Resource    resources/common.resource
Suite Setup    Log Test Start
Suite Teardown    Log Test End

*** Test Cases ***
Resource Keyword Test
    Log    Running test logic
```

Expected Result: The resource keywords are available to the suite and setup/teardown keywords run at the appropriate lifecycle points.

Conclusion: Resource files and lifecycle keywords improve maintainability and consistent test preparation/cleanup.

Experiment 10 – Page Object and Gherkin-Style Readability

Aim: To represent a page through reusable keywords and write a readable test flow resembling Gherkin.

Procedure

1. Create page-level keywords for navigation and validation.
2. Keep selectors inside reusable keywords.
3. Write the test as a readable business flow.
4. Separate page behavior from test intent.
5. Execute the test and review the report.

Robot Framework Source

```
*** Settings ***
Library    SeleniumLibrary

*** Keywords ***
Given User Opens Selenium Website
    Open Browser    https://www.selenium.dev/    chrome

When User Opens Documentation
    Click Link    Documentation

Then Documentation Page Is Displayed
    Wait Until Page Contains    Selenium Documentation

*** Test Cases ***
Documentation Navigation
    Given User Opens Selenium Website
    When User Opens Documentation
    Then Documentation Page Is Displayed
    [Teardown]    Close All Browsers
```

Expected Result: The test reads as a business-oriented Given/When/Then flow while browser details remain inside keywords.

Conclusion: Page-level keywords and Gherkin-style readability make Robot Framework suites easier to understand and maintain.

Overall Module 4 Lab Conclusion

The practical work follows the Robot Framework portion of the prescribed course sheet. It covers basic syntax and keywords, environment setup, SeleniumLibrary browser automation, keyword-driven and data-driven testing, command-line execution, remote-browser execution patterns, Jenkins scheduling, resource files, setup/teardown, Page Object concepts and Gherkin-style readability.

Syllabus Area	Experiments
Introduction / Setup	1–2
Keyword & Data-Driven Test Cases	3–4
Command Line / Browser / Execution	5–6
Sauce Labs / Jenkins	7–8
User Keywords / Resources / Page Objects / Gherkin	9–10